Individual Contribution Reflection

Tom Tian

COMP4347 / COMP5347 Assignment 2 - MERN Quiz Game

Primary subsystem: integration, validation, robustness, shared shell, documentation, and tests

Subsystem ownership. My primary responsibility was Subsystem D: the cross-cutting integration layer that made the independently owned authentication, quiz, admin, and Review Mode work behave as one coherent MERN application. I implemented and hardened shared Express app wiring, response envelopes, error handling, rate-limit responses, Swagger/Postman documentation, frontend API integration, theme/shared shell behaviour, and regression tests. I deliberately did not take over Auth, Quiz/Review/Leaderboard business logic, or Admin CRUD ownership; my work focused on the contracts where those parts meet.

1 Implementation Against the Rubric

The rubric rewards correct architecture, protected APIs, validation, security controls, usable React integration, documentation, and evidence of individual work. My contribution targeted those integration and robustness criteria rather than adding speculative product features.

- **Backend integration:** `backend/src/app.js` wires security middleware, route mounting, Swagger, 404 handling, and the central `errorHandler`.
- **Response contract:** `backend/src/utils/responseEnvelope.js` standardises `{ success, data?, error? }`, with tests covering success and failure paths.
- **Validation and robustness:** shared rate-limit handlers return envelope-compatible `429` responses; final Swagger fixes align documented `401/403/429` outcomes with route middleware.
- **Frontend shell:** `App.jsx`, `api.js`, `main.jsx`, and `ThemeContext.jsx` connect routing, protected UI flow, API calls, and persisted theme behaviour.
- **Documentation and QA:** Swagger, Postman, README evidence, and Jest/Supertest tests make the submitted behaviour auditable by markers.

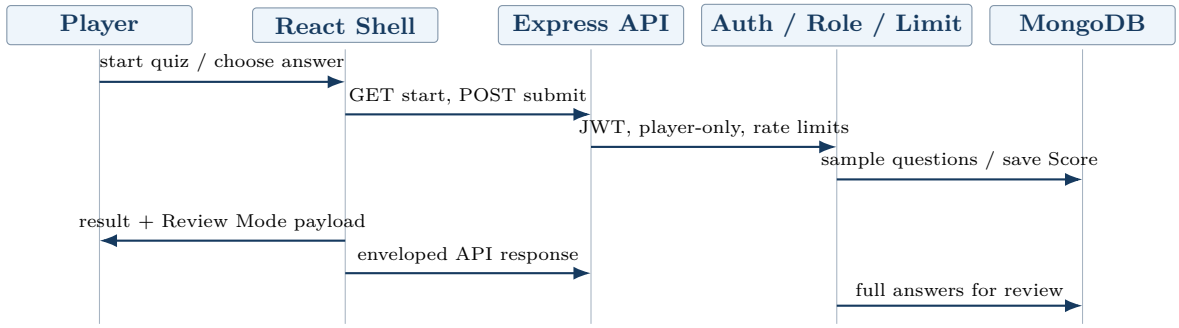
2 Major Technical Challenge

The hardest integration issue was maintaining one answer contract across shuffled quiz options, scoring, persistence, and Review Mode. `/api/quiz/start` returns shuffled options without exposing `correctAnswer`; the frontend submits the visible option index; the backend scores against the same deterministic shuffled order; and Review Mode must later display the selected and correct answers in that same order.

I found a backend API test that still assumed the raw seed answer index, which could create false failures or hide a real drift between start, submit, save, and review behaviour. I corrected `backend/src/tests/api.test.js` so the test first calls `/api/quiz/start`, locates the correct visible option, submits through `/api/quiz/submit`, and verifies the saved review payload. Commit `bba8b06` is the direct evidence.

This fix supports the approved Review Mode variation without changing scoring rules. It validates the learning-feedback contract instead of inventing a new quiz mechanic.

3 Mermaid-Style Sequence Diagram



4 Git Commit History Analysis

The commits below show my contribution pattern: establish shared infrastructure, document and verify integration contracts, then harden the final submission surface. The hashes link to GitHub Enterprise for marker verification.

Hash	Commit focus	Reflection on contribution
b176192	workspace scaffolding	Set up the project structure so backend, frontend, scripts, and shared checks could evolve together.
90bf1d2	scaffold dependencies	Aligned root, backend, and frontend scripts so reviewers could install, run, test, and build consistently.
b6d0ffe	backend foundation	Established the Express foundation later used by feature owners and integration tests.
57ebccb	shared rate limiters	Added reusable login and quiz-submit throttling with envelope-compatible failure responses.
d7c20c6	frontend providers	Integrated shared providers needed for routing, auth-aware API use, and theme behaviour.
51c7e39	shared app shell	Created the application shell tying public, player, and admin flows into one navigable app.
8501ac2	Swagger docs	Added self-hosted API documentation required by the assignment and useful for contract review.
7acd98d	envelope/error tests	Verified response-envelope and error-handler behaviour rather than relying on manual inspection.
03a5d8e	architecture docs	Documented integration decisions and system architecture for marker review and team alignment.
2bda2bb	security/API evidence	Recorded validation and security evidence around JWT, rate limiting, and API behaviour.
6686eae	D docs/tests	Expanded seed, API, Postman, and regression coverage around the shared delivery surface.
972fc77	final QA alignment	Reconciled D-owned tests and documentation with the real API surface before review.
bba8b06	shuffled answer tests	Fixed a test-contract bug so Review Mode scoring and assertions match visible shuffled options.
4a8fe22	leaderboard API docs	Aligned Swagger with the authenticated, player-only leaderboard route.
d0d07f6	quiz error responses	Completed route, middleware, and Swagger consistency for `403` and `429` outcomes.

5 Review Mode Design Reflection and Final Evidence

Review Mode was a sound variation because it improves educational value without changing the core scoring rule of +1 per correct answer. My integration work supported that design by making the completed attempt store enough answer data to reconstruct what the player saw and selected. The main design risk was contract drift: if shuffled options, submitted indexes, saved answers, and review rendering disagreed, Review Mode could look correct while explaining the wrong answer. The final API test and Swagger reconciliation therefore protected both correctness and auditability.

Final verification evidence: `npm test -prefix backend` passed 6 suites / 22 tests; `npm run build -prefix frontend` passed; `python3 -m json.tool docs/postman-collection.json >/dev/null` passed; `git diff -check` passed; root, backend, and frontend `npm audit` reported 0 vulnerabilities.